Проектирование ИС: сервис студенческих знакомств CAMPUS (IDEF0, IDEF1X, DFD, IDEF3, FURPS+, UML)

13 ноября 2026 · Команда CAMPUS · УрФУ · расширенный уровень · редакция 2

1. Описание предметной области

Описание идёт от общего к частному: сначала цель сервиса, затем направления его деятельности, затем участники, объекты и правила, которые действуют внутри этих направлений.

Цель. CAMPUS — сервис знакомств и совместных активностей для студентов вузов Екатеринбурга. Его задача — помочь студентам одного кампуса находить друг друга и встречаться офлайн: на свидании, за кофе или на мероприятии. Пилот рассчитан на один вуз (УрФУ) и один сервер; оплата работает в тестовом режиме, цифры нагрузки — допущения.

Направления деятельности. Чтобы достичь цели, сервис делает три вещи:

1. **Ведёт сообщество студентов** — допускает только совершеннолетних студентов с подтверждённой почтой вуза, хранит их анкеты и поддерживает безопасность: жалобы, личные блокировки, решения модератора.
2. **Организует знакомства и встречи** — то, ради чего студент приходит: лента анкет, взаимные симпатии, переписка, предложения встреч и мероприятия с партнёрами-площадками.
3. **Принимает оплату и ведёт отчётность** — обеспечивает работу сервиса: оплата билетов и подписки Premium через платёжный сервис и отчёты для команды.

Как сервис устроен сейчас. Все правила выполняет один сервер (API на NestJS с базой PostgreSQL и Redis). К нему подключаются клиенты: сайт и мобильное приложение для Android и iOS работают; Telegram-бот с Mini App запланирован. Студент входит по одноразовому коду на корпоративную почту вуза — этот же код подтверждает, что он студент.

Участники (роли):

Роль	Что делает в системе
Студент	Входит по почте вуза, заполняет анкету, ставит лайки, переписывается, предлагает встречи, жалуется и блокирует, записывается на мероприятия, оплачивает билеты и Premium
Модератор	Рассматривает жалобы, принимает решение: отклонить, предупредить или заблокировать

Роль	Что делает в системе
Организатор (команда CAMPUS)	Согласует площадку с партнёром, публикует мероприятие, отмечает вход по QR-билету
Партнёр (площадка)	Предоставляет место и условия, получает список участников
Администратор (команда CAMPUS)	Настраивает тарифы и правила, смотрит отчёты
Платёжный сервис (внешняя система)	Принимает оплату и присылает уведомление о результате
Система (по расписанию)	Каждые 5 минут снимает неоплаченные брони

Основные объекты (сущности): пользователь, анкета, фото, интерес, блокировка, жалоба, лайк, совпадение, сообщение, встреча, партнёр, мероприятие, билет, платёж, подписка Premium.

Бизнес-правила (по направлениям):

1. Сообщество: пользоваться сервисом можно только с 18 лет; возраст считается по дате рождения при регистрации, вместе с ней даётся согласие на обработку персональных данных.
2. Сообщество: отметка «студент подтверждён» ставится только после ввода кода, отправленного на почту в домене вуза. Код живёт 10 минут; после 3 ошибок почта блокируется на 15 минут.
3. Сообщество: анкета, на которую пожаловались три разных пользователя, скрывается из ленты до решения модератора (не дольше 24 часов). Личная блокировка действует сразу: пользователи перестают видеть друг друга в ленте, симпатиях и чатах.
4. Сообщество: студент может скрыть анкету от студентов своего института или временно убрать её из ленты.
5. Знакомства: в ленту попадает только подтверждённая анкета с 1–6 фото и 3–10 интересами.
6. Знакомства: чат открывается только после взаимного лайка. Бесплатно — до 30 лайков в сутки, с Premium — без лимита.
7. Знакомства: место на мероприятии бронируется на 30 минут; вход по QR-билету отмечается один раз.
8. Оплата: билет и Premium действуют только после подтверждения оплаты платёжным сервисом. Premium действует 30 дней и стоит 349 ₽.
9. Точная геолокация не собирается и не хранится.

1.1. Дерево задач



Дерево задач сервиса CAMPUS

Логическое описание. Дерево построено сверху вниз — от глобальной цели к локальным задачам. Корень (уровень 0) — цель сервиса: помочь студентам знакомиться и встречаться. От корня идёт один ствол, который делится на три направления (уровень 1). Каждое направление отвечает на свой вопрос, и все функции внутри него служат одной цели:

- **1. Сообщество студентов** — «кто допускается в сервис и как в нём остаётся безопасно». Без учётной записи, подтверждённого статуса и анкеты остальные направления работать не могут, поэтому направление первое. В него же входит модерация: она решает, кто остаётся в сообществе.
- **2. Знакомства и встречи** — «что студент получает от сервиса». Центральное направление: использует анкеты из направления 1 и выдаёт совпадения, переписку, встречи и мероприятия.
- **3. Оплата и отчётность** — «на чём держится работа сервиса». Обеспечивающее направление: проверяет оплату для мероприятий и Premium и считает отчёты по уже накопленным данным.

Уровень 2 — функции внутри направлений (11...33). Функции, которые сами состоят из нескольких шагов, раскрыты на уровне 3: модерация (14 → 141–143) и мероприятия (25 → 251–254). Остальные функции уже достаточно конкретны и являются листьями. Так глубина дерева отражает сложность работы: дерево не «кустится» от корня десятком равноправных ветвей, а растёт ярусами — каждое следующее разбиение уточняет предыдущее.

Нумерация. Используется многозначная (десятичная) нумерация: уровень 1 — 1…3, уровень 2 — 11…33, уровень 3 — 141…254. Каждая следующая цифра — номер узла внутри родителя. Номер узла совпадает с номером блока IDEF0 без буквы A: функция 25 — блок A25, задача 253 — блок A253. Такая нумерация принята в стандарте IDEF0 (Р 50.1.028-2001, FIPS 183) и позволяет пройти от листа дерева к диаграмме и обратно. Каждое разбиение содержит от 3 до 5 узлов — столько же, сколько блоков допускает стандарт на одной диаграмме.

Правило «один лист — одна ветвь». Каждая задача встречается в дереве один раз. Вход есть только в листе 11, проверка оплаты — только в листе 32; направление 2 лишь формирует заказ билета (253) и получает статус оплаты, но само оплату не проверяет. У каждого листа указано хранилище данных («→ D…»): таблицы хранилищ и есть база данных (раздел 8) и хранилища DFD (раздел 9).

1.2. Результаты задач и движение информации

Для каждого листа указано, какая информация в нём формируется (не документ, а конкретные данные), кому она передаётся дальше и где хранится. Эти результаты становятся стрелками IDEF0 и потоками DFD.

Задача	Результат (информация)	Куда передаётся	Хранилище
11 Регистрация и вход	Учётная запись, сессия (токен), статус пользователя	12, 21	D1.1 Пользователи
12 Подтверждение статуса студента	Отметка «студент подтверждён»	13, 21	D1.1 Пользователи
13 Заполнение анкеты	Анкета: фото, интересы, цель знакомства	21	D1.2 Анкеты, D1.3 Фото
141 Приём жалоб и блокировок	Жалоба в очереди; скрытие анкеты при 3 жалобах; личная блокировка	142, 21, 23	D1.4 Жалобы, D1.5 Блокировки
142 Рассмотрение жалобы	Решение модератора	143	D1.4 Жалобы
143 Применение решения	Статус пользователя, уведомления	11, 21, студентам	D1.1 Пользователи
21 Формирование ленты	Лента анкет для студента	22, студенту	— (читает D1.2, D1.5, D2.1)
22 Лайки и совпадения	Лайк, совпадение; оценённые анкеты	23, 21, 33	D2.1 Лайки и совпадения
23 Переписка	Сообщения; жалоба из чата	24, 141	D2.2 Сообщения
24 Согласование встречи	Предложение и ответ: место, время, статус	студентам, 33	D2.3 Встречи
251 Согласование с партнёром	Условия площадки: дата, вместимость, цена	252	D2.4 Партнёры
252 Публикация мероприятия	Карточка мероприятия, афиша	253, студентам	D2.5 Мероприятия

Задача	Результат (информация)	Куда передаётся	Хранилище
253 Запись участников	Бронь (заказ билета); билет с QR-кодом	31, 254, партнёру	D2.6 Билеты
254 Отметка входа по QR	Отметка «пришёл»; список пришедших	организатору, 33	D2.6 Билеты
31 Создание платежа	Платёж «ожидает», ссылка на оплату	32, студенту	D3.1 Платежи
32 Проверка оплаты	Статус оплаты; подписка Premium	253, 22, 33	D3.1, D2.6, D3.2 Подписки
33 Формирование отчётов	Отчёт за период и регистрации по дням	команде, партнёрам	— (читает D1–D3)

Видно, как информация возвращается между направлениями. Заказ билета уходит из листа 253 (направление 2) в лист 31 (направление 3); проверка оплаты (32) возвращает статус обратно в лист 253, и только тогда студент получает билет. Так же жалоба из чата (23) уходит в модерацию (141), а решение модератора (143) возвращается в ленту (21) и ко входу (11).

1.3. Матрица ответственности

Матрица построена по листьям дерева и показывает, кто выполняет каждую задачу. Обозначения: **И** — исполняет, **У** — участвует (предоставляет данные), **К** — контролирует или утверждает, **Н** — получает результат. Столбцы с буквой И становятся актёрами диаграммы вариантов использования (раздел 12.1): «Студент», «Модератор» и «Платёжный сервис» переходят как есть; «Организатор» и «Админ» — сотрудники команды сервиса, поэтому на UML они объединены в актёра «Команда CAMPUS»; «Система» — это сам сервис (граница системы на UML), а её задачу по расписанию (снятие неоплаченных броней в листе 253) запускает системный актёр «Таймер». «Партнёр» только участвует (У) и получает результат (Н), поэтому актёром UML не стал.

Задача	Студент	Модератор	Организатор	Партнёр	Админ	Платёжный сервис	Система
11 Регистрация и вход	И						У
12 Подтверждение статуса	И						У
13 Заполнение анкеты	И						У
141 Приём жалоб и блокировок	И	Н					У
142 Рассмотрение жалобы		И			К		
143 Применение решения	Н	И					У
21 Формирование ленты	Н						И
22 Лайки и совпадения	И						У
23 Переписка	И						У
24 Согласование встречи	И						У

Задача	Студент	Модератор	Организатор	Партнёр	Админ	Платёжный сервис	Система
251 Согласование с партнёром			И	У	К		
252 Публикация мероприятия	Н		И				У
253 Запись участников	И		Н	Н			У
254 Отметка входа по QR	У		И	Н			У
31 Создание платежа	И					У	У
32 Проверка оплаты	Н		Н			У	И
33 Формирование отчётов			Н	Н	Н		И

Из матрицы следует: студент исполняет 9 задач из 17, поэтому интерфейс строится вокруг него; модератор нужен только в функции 14; организатор — только в мероприятиях (25); платёжный сервис участвует в двух задачах направления 3 и в программе становится внешней системой с API; задачи 21, 32 и 33 выполняются автоматически, без участия людей.

2. Цель, точка зрения и границы модели

Параметр	Значение
Название модели	«Деятельность сервиса знакомств CAMPUS»
Нотации	IDEF0 (функциональная модель), IDEF1X (модель данных), DFD (потоки данных, Гейн–Сарсон), IDEF3 (сценарии процессов), UML 2.5
Цель моделирования	Описать функции сервиса и потоки информации между ними, чтобы определить требования к ИС, сформировать техническое задание и проверить по модели реализацию
Точка зрения	Команда (администратор) сервиса
В границах модели	Вход и подтверждение студента, анкеты, модерация и блокировки, лента, лайки, совпадения, чат и встречи, мероприятия с партнёрами и вход по QR, оплата билетов и Premium, отчёты
За границами модели	Реальный эквайринг (только тестовый режим), публикация в магазинах приложений, рекомендации на машинном обучении, геолокация, реклама, бухгалтерия партнёров

Нумерация блоков — по стандарту IDEF0: A-0 (контекст), A1…A3 (направления), A11…A33 (функции), A141…A143 и A251…A254 (задачи третьего уровня). Стрелки ICOM: **I** — вход (слева), **C** — управление (сверху), **O** — выход (справа), **M** — механизм (снизу).

3. Предположения по сценариям и нагрузке

Семь сценариев покрывают все блоки модели; основная нагрузка — вечерний просмотр ленты (функция A21).

3.1. Сценарии

№	Сценарий	Участник	Ход	Результат	Блоки
C1	Регистрация и подтверждение	Новый студент	Дата рождения (18+) и согласие → вуз → почта вуза → код из письма → анкета → фото	Анкета в ленте	A11–A13
C2	Знакомство (основной)	Студент	Лента → лайк → взаимный лайк → совпадение → чат → предложение встречи	Согласованная встреча	A21–A24
C3	Жалоба	Студент, модератор	Жалоба → при 3 жалобах анкета скрыта → рассмотрение → решение	Статус пользователя	A141–A143
C4	Мероприятие	Организатор, партнёр	Согласование площадки → публикация → запись → вход по QR	Мероприятие с участниками	A251–A254
C5	Покупка билета	Студент, платёжный сервис	Бронь на 30 мин → платёж → уведомление → проверка → билет с QR или снятие брони	Оплаченный билет	A253, A31, A32
C6	Покупка Premium	Студент	Оплата 349 ₽ → проверка → Premium на 30 дней → лимит лайков снят	Активный Premium	A31, A32, A22
C7	Отчёт	Администратор	Выбор периода → регистрации, совпадения, встречи, билеты, выручка; регистрации по дням	Отчёт за период	A33

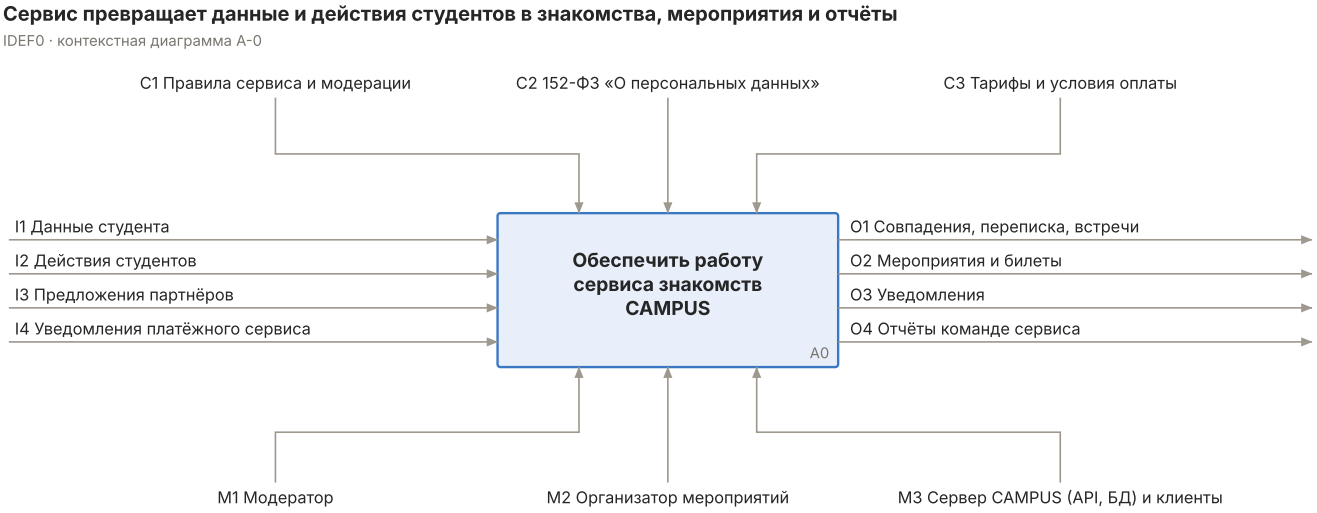
3.2. Предположения о нагрузке

Учебный масштаб: пилот в одном вузе, один сервер; все числа — допущения.

Показатель	Значение	Как получено
Зарегистрированные студенты	1 000	допущение для пилота в УрФУ
Активные за день (DAU)	300	30% зарегистрированных
Просмотры анкет в сутки	≈ 12 000	300 × 40 анкет
Лайки в сутки	≈ 3 000	25% просмотров
Совпадения в сутки	≈ 150	5% лайков взаимны
Сообщения в сутки	≈ 3 000	300 активных пар × 10 сообщений
Одновременно онлайн в пик (21:00–23:00)	≈ 60	20% DAU
Запросы к API в пик	≈ 17 в секунду	60 пользователей; чат и оплата опрашивают сервер раз в 3–5 с
Объём фото	≈ 1 ГБ	1 000 × 3 фото × 300 КБ
Оплаты	≈ 5 в сутки	Premium 3% (30 в месяц) + 4 мероприятия × 30 билетов

Выводы для проектирования: нагрузка на один сервер небольшая; узкое место не производительность, а плотность аудитории (нужны анкеты в ленте). Самый частый запрос — чтение ленты, поэтому на таблицах анкет и лайков нужны индексы, а лента отдаётся страницами. Чат пока обновляется опросом сервера — это главный источник лишних запросов; при росте его заменят на WebSocket.

4. Контекстная диаграмма A-0



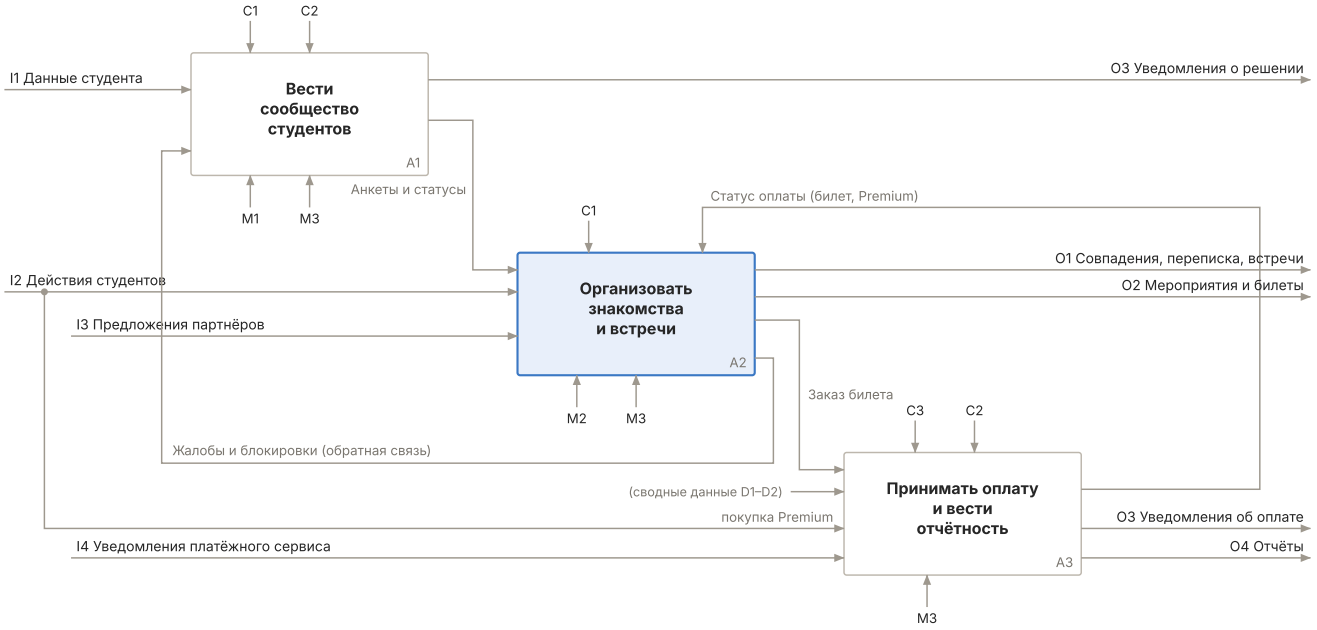
IDEF0 A-0 · контекстная диаграмма · точка зрения: команда сервиса

Логическое описание. Контекстная диаграмма — самый глобальный уровень модели: весь сервис показан одним блоком A0 вместе с внешним миром. Слева входит то, что сервис перерабатывает: данные студента для входа и анкеты (I1), действия студентов — лайки, сообщения, жалобы, заявки и покупки (I2), предложения партнёров-площадок (I3) и уведомления платёжного сервиса (I4). Сверху деятельность ограничивают правила сервиса и модерации (C1), закон о персональных данных (C2) и тарифы (C3). Снизу — кто выполняет работу: модератор (M1), организатор мероприятий (M2), сервер CAMPUS с базой данных и клиентами — сайтом и мобильным приложением (M3). Справа — результат: совпадения, переписка и встречи (O1), мероприятия и билеты (O2), уведомления (O3) и отчёты команде (O4).

5. Декомпозиция A0 — три направления

Жалобы и статус оплаты возвращаются в предыдущие направления

IDEF0 · декомпозиция A0 на три направления, обратные связи A2 → A1 (жалобы) и A3 → A2 (статус оплаты)



IDEF0 A0 · декомпозиция на три направления

Логическое описание. Первый уровень декомпозиции повторяет ствол дерева задач: три блока — три направления. Блоки расположены лесенкой по доминированию. A1 «Вести сообщество студентов» выпускает анкеты и статусы пользователей, без которых A2 не может построить ленту. A2 «Организовать знакомства и встречи» — главный блок: выдаёт совпадения, переписку и встречи (O1), мероприятия и билеты (O2). Из A2 обратно в A1 идут жалобы и блокировки — первая петля обратной связи (по входу, линия проходит под блоками). Заказ билета из A2 уходит в A3 «Принимать оплату и вести отчётность», а статус оплаты возвращается в A2 как управление (бронь становится билетом, Premium снимает лимит лайков) — вторая петля, линия проходит над блоками. Для отчётов A3 читает сводные данные из хранилищ D1–D2 — туннельная стрелка в скобках: на IDEF0 нет хранилищ, а путь данных виден на DFD (раздел 9.4).

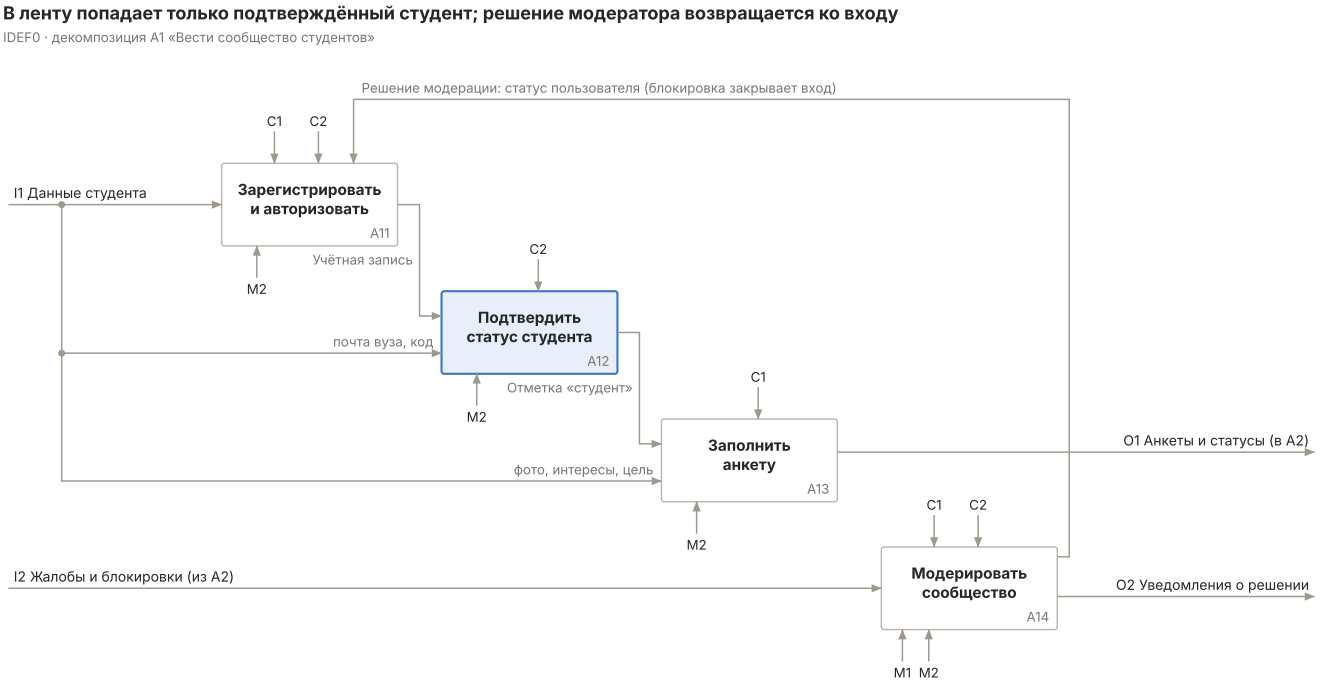
Блок	Функция	Вход (I)	Управление (C)	Выход (O)	Механизм (M)
A1	Вести сообщество студентов	I1 Данные студента; жалобы и блокировки (из A2)	C1, C2	Анкеты и статусы (в A2), O3 Уведомления о решении	M1, M3
A2	Организовать знакомства и встречи	Анкеты и статусы (из A1), I2 Действия студентов, I3 Предложения партнёров	C1, статус оплаты (из A3)	O1, O2, заказ билета (в A3), жалобы (в A1)	M2, M3

Блок	Функция	Вход (I)	Управление (C)	Выход (O)	Механизм (M)
A3	Принимать оплату и вести отчётность	Заказ билета (из A2), покупка Premium (I2), I4, (сводные данные D1–D2)	C3 Тарифы, C2	Статус оплаты (в A2), O3, O4 Отчёты	M3

6. Декомпозиция второго уровня (A1–A3)

Каждое направление развёрнуто на функции. Стрелки на границе повторяют стрелки родительского блока на A0, но коды ICOM у них локальные — расшифровка в подписи под каждой диаграммой. Управление и механизмы показаны короткими отрезками у блоков, чтобы линии не пересекались.

6.1. Декомпозиция A1 «Вести сообщество студентов»



IDEF0 A1 · декомпозиция «Вести сообщество студентов»

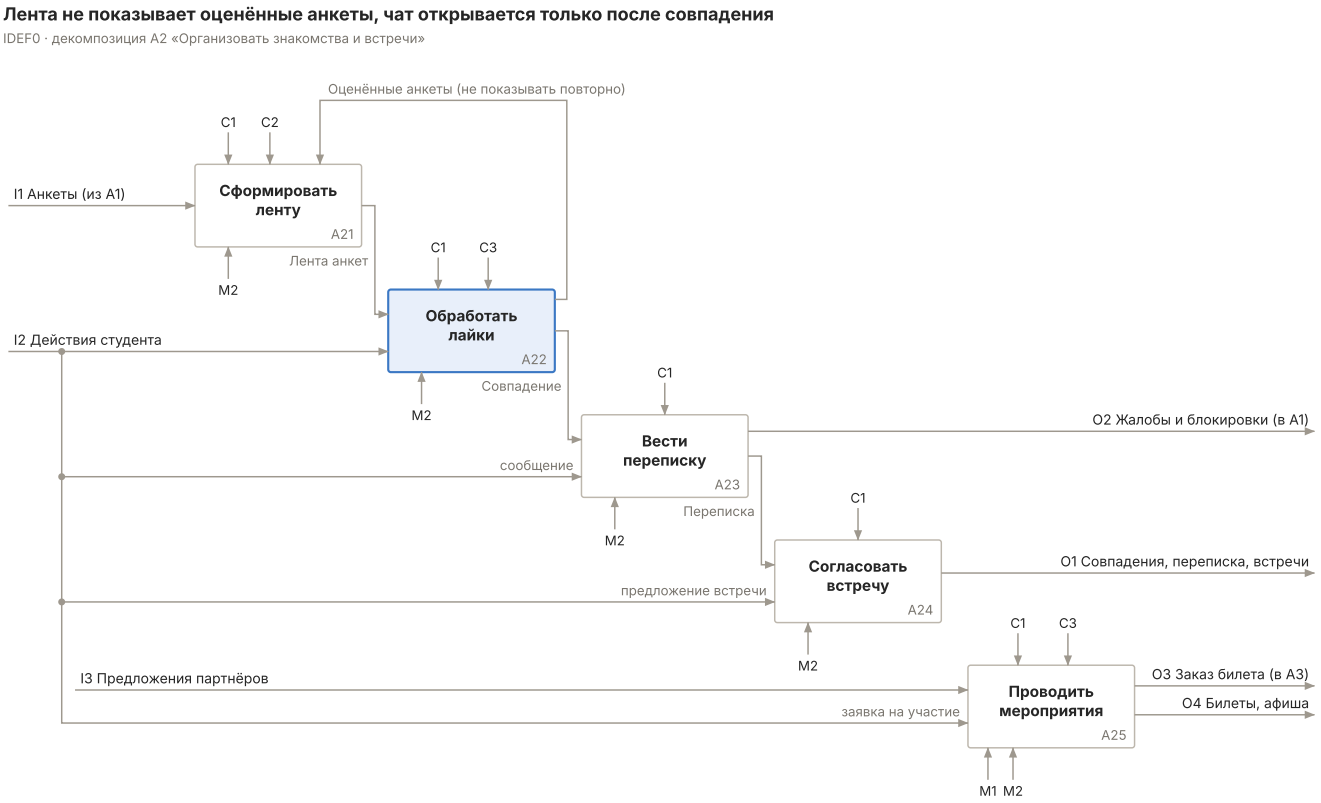
Узел A1 · I1 — данные студента (дата рождения, согласие, почта вуза, код, фото, интересы) · I2 — жалобы и блокировки (из A2) · C1 — правила сервиса · C2 — 152-ФЗ · O1 — анкеты и статусы (в A2) · O2 — уведомления о решении · M1 — модератор · M2 — сервер CAMPUS

Логическое описание. Блок покрывает сценарии C1 и C3. A11 превращает дату рождения и согласие в учётную запись — только для 18+ (C1) и с согласием на обработку персональных данных (C2). В A12 студент получает на почту вуза одноразовый код и вводит его; после этого в учётной записи ставится отметка «студент». В реализации оба шага выполняются одним кодом: он и впускает студента, и подтверждает почту. Только с отметкой A13 принимает фото и интересы и выпускает анкету (O1). Функция A14 «Модерировать сообщество» получает жалобы и блокировки из направления 2 и возвращает решение модератора вверх, в A11, как

управление: заблокированный пользователь больше не войдёт. Сама A14 раскрыта на третьем уровне (раздел 7.1).

Блок	Вход	Управление	Выход	Механизм
A11 Зарегистрировать и авторизовать	I1 дата рождения, согласие, почта	C1 (18+), C2 (согласие), решение модерации (из A14)	Учётная запись	M2
A12 Подтвердить статус студента	Учётная запись, I1 код из письма	C2	Отметка «студент»	M2
A13 Заполнить анкету	Отметка «студент», I1 фото, интересы, цель	C1 (1–6 фото, 3–10 интересов)	O1 Анкеты и статусы	M2
A14 Модерировать сообщество	I2 Жалобы и блокировки	C1, C2	Решение модерации (в A11), O2 Уведомления	M1, M2

6.2. Декомпозиция A2 «Организовать знакомства и встречи»



IDEF0 A2 · декомпозиция «Организовать знакомства и встречи»

Узел A2 · I1 — анкеты и статусы (из A1) · I2 — действия студента · I3 — предложения партнёров · C1 — правила (лимит 30 лайков, публичное место встречи, вместимость) · C2 — решение модерации · C3 — статус оплаты (из A3) · O1 — совпадения, переписка, встречи · O2 — жалобы и блокировки (в A1) · O3 — заказ билета (в A3) · O4 — билеты и афиша · M1 — организатор · M2 — сервер CAMPUS

Логическое описание. Блок покрывает сценарии C2 и C4. Первые четыре функции — цепочка знакомства. A21 строит ленту из анкет (I1): только подтверждённые анкеты, без скрытых

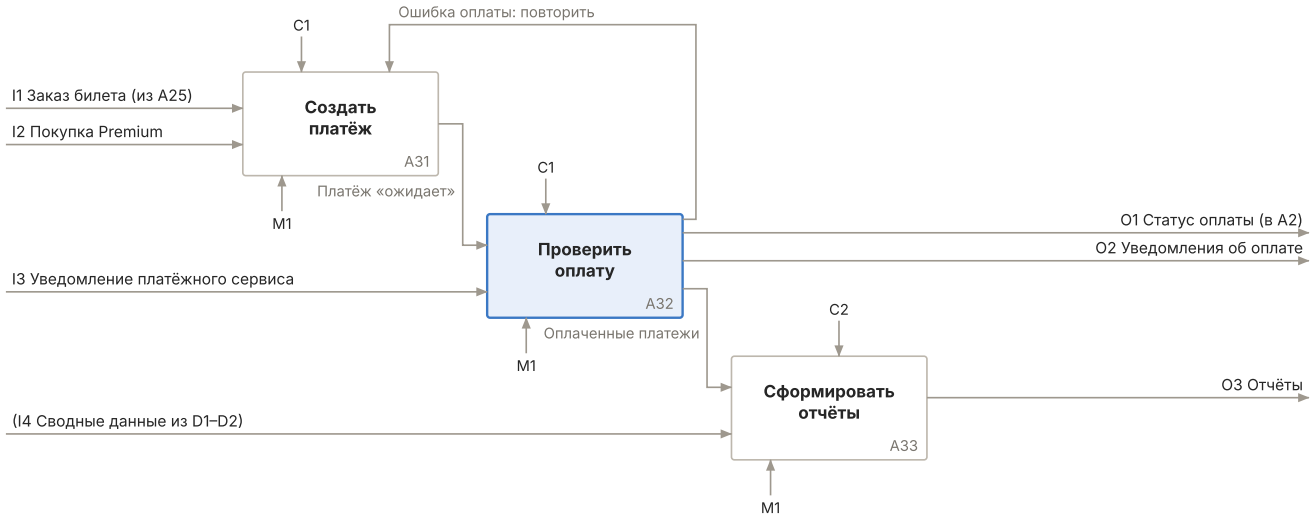
модератором (C2), заблокированных лично и студентов своего института, если они это запретили (C1). A22 обрабатывает лайк: проверяет лимит (C1), который снимает Premium (C3), и при взаимном лайке выдаёт совпадение. Оценённые анкеты возвращаются в A21 — обратная связь, чтобы одна анкета не показывалась дважды. Совпадение открывает чат в A23, откуда может уйти жалоба или блокировка (O2). В A24 студенты согласуют встречу; итог — O1. Пятая функция A25 «Проводить мероприятия» — вторая форма знакомства: она получает предложения партнёров и заявки студентов, отправляет заказ билета на оплату (O3) и закрепляет место по статусу оплаты (C3). A25 раскрыта на третьем уровне (раздел 7.2).

Блок	Вход	Управление	Выход	Механизм
A21 Сформировать ленту	I1 Анкеты	C1, C2, оценённые анкеты (из A22)	Лента анкет	M2
A22 Обработать лайки	Лента, I2 лайк	C1 (30 в сутки), C3 Premium	Совпадение, оценённые анкеты	M2
A23 Вести переписку	Совпадение, I2 сообщение	C1	Переписка, O2 Жалобы и блокировки	M2
A24 Согласовать встречу	Переписка, I2 предложение	C1 (публичное место)	O1 Встречи	M2
A25 Проводить мероприятия	I3 Предложения партнёров, I2 заявки	C1 (вместимость, 18+), C3 статус оплаты	O3 Заказ билета, O4 Билеты и афиша	M1, M2

6.3. Декомпозиция A3 «Принимать оплату и вести отчётность»

Только подтверждённая платёжным сервисом оплата меняет статус билета и Premium

IDEF0 · декомпозиция A3 «Принимать оплату и вести отчётность»



IDEF0 A3 · декомпозиция «Принимать оплату и вести отчётность»

Узел A3 · I1 — заказ билета (из A25) · I2 — покупка Premium · I3 — уведомление платёжного сервиса · (I4) — сводные данные из D1–D2 (туннельная стрелка) · C1 — тарифы (349 ₽, бронь 30 минут) · C2 — 152-ФЗ · O1 — статус оплаты (в A2) · O2 — уведомления об оплате · O3 — отчёты · M1 — сервер CAMPUS

Логическое описание. Блок покрывает сценарии C5–C7. A31 по тарифу (C1) создаёт платёж со статусом «ожидает» и получает ссылку на оплату; повторный запрос с тем же ключом идемпотентности возвращает тот же платёж. A32 сверяет платёж с уведомлением платёжного сервиса (I3): при успехе выдаёт статус оплаты (O1), который уходит в A2, и уведомление студенту (O2). При ошибке платёж возвращается в A31 на повторную оплату — обратная связь по управлению. A33 стоит последним: считает выручку только по проверенным платежам и отдельно берёт сводные данные за период из D1–D2 (I4) — регистрации, совпадения, встречи, билеты. Отчёты (O3) выпускаются только в обезличенном виде (C2).

Блок	Вход	Управление	Выход	Механизм
A31 Создать платёж	I1 Заказ билета, I2 Покупка Premium	C1 Тарифы, ошибка оплаты (из A32)	Платёж «ожидает»	M1
A32 Проверить оплату	Платёж, I3 Уведомление	C1 (бронь 30 мин)	O1 Статус оплаты, O2 Уведомления, оплаченные платежи	M1
A33 Сформировать отчёты	Оплаченные платежи, (I4) сводные данные D1–D2	C2 (обезличенные данные)	O3 Отчёты	M1

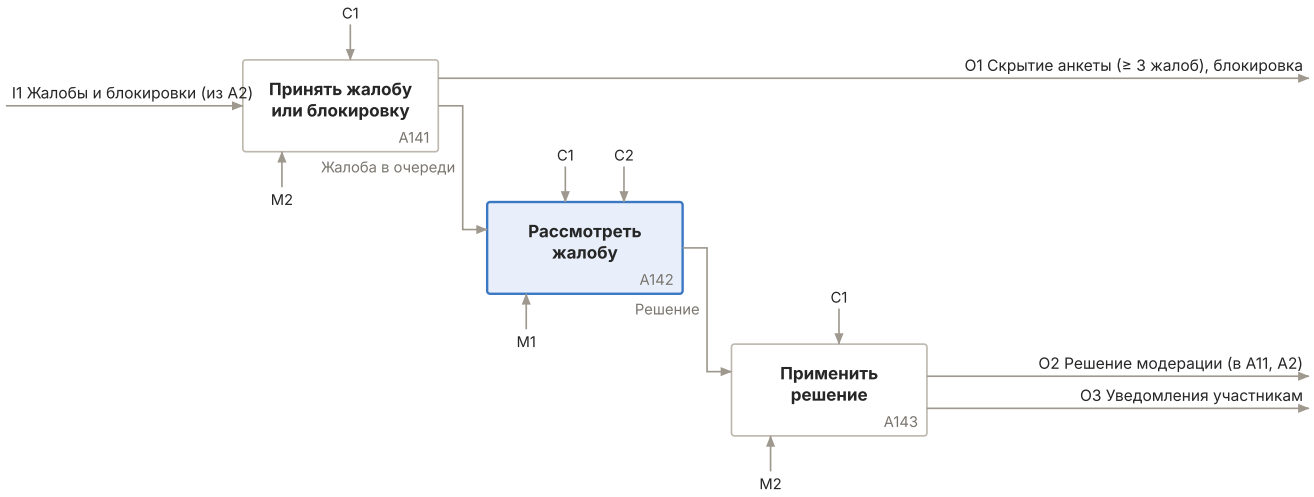
7. Декомпозиция третьего уровня (A14, A25)

Две функции второго уровня состоят из нескольких самостоятельных шагов с разными исполнителями, поэтому раскрыты ещё на один уровень — так же, как в дереве задач.

7.1. Декомпозиция A14 «Модерировать сообщество»

Личная блокировка действует сразу, решение по жалобе принимает только модератор

IDEF0 · декомпозиция A14 «Модерировать сообщество»



IDEF0 A14 · декомпозиция «Модерировать сообщество»

Узел A14 · I1 — жалобы и блокировки (из A2) · C1 — правила модерации · C2 — 152-ФЗ · O1 — скрытие анкеты и личная блокировка (в A21) · O2 — решение модерации (в A11, A2) · O3 — уведомления · M1 — модератор · M2 — сервер CAMPUS

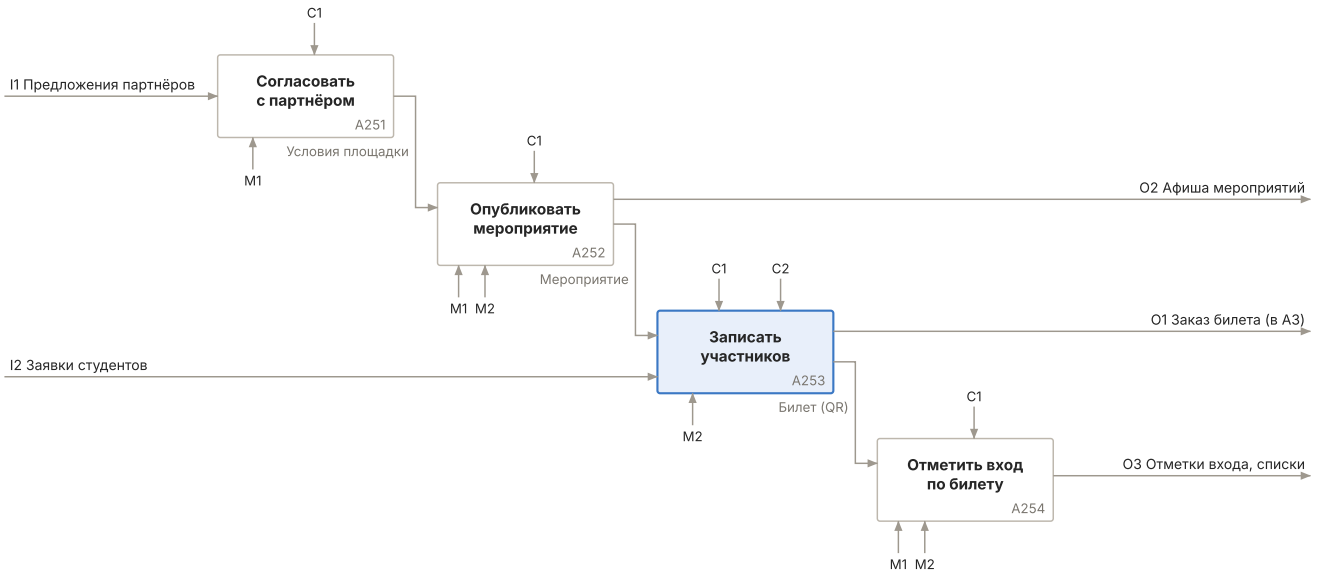
Логическое описание. A141 принимает жалобу или личную блокировку. Блокировка действует сразу и не требует модератора: пользователи перестают видеть друг друга (O1). Жалоба ставится в очередь; если на анкету пожаловались три разных пользователя (C1), анкета сразу скрывается из ленты — до решения человека. A142 — единственная работа модератора (M1): он видит жалобу и до 20 последних сообщений переписки, на которую пожаловались (C2 ограничивает доступ к персональным данным), и принимает решение. A143 применяет решение: меняет статус пользователя (O2, уходит в A11 и в ленту) и уведомляет участников (O3).

Блок	Вход	Управление	Выход	Механизм
A141 Принять жалобу или блокировку	I1 Жалоба, блокировка	C1 (3 жалобы → скрыть)	Жалоба в очереди, O1 скрытие, блокировка	M2
A142 Рассмотреть жалобу	Жалоба в очереди	C1, C2 (доступ только к переписке по жалобе)	Решение	M1
A143 Применить решение	Решение	C1	O2 Решение модерации, O3 Уведомления	M2

7.2. Декомпозиция A25 «Проводить мероприятия»

Место закрепляется только после оплаты, а вход по QR-билету отмечается один раз

IDEF0 · декомпозиция A25 «Проводить мероприятия»



IDEF0 A25 · декомпозиция «Проводить мероприятия»

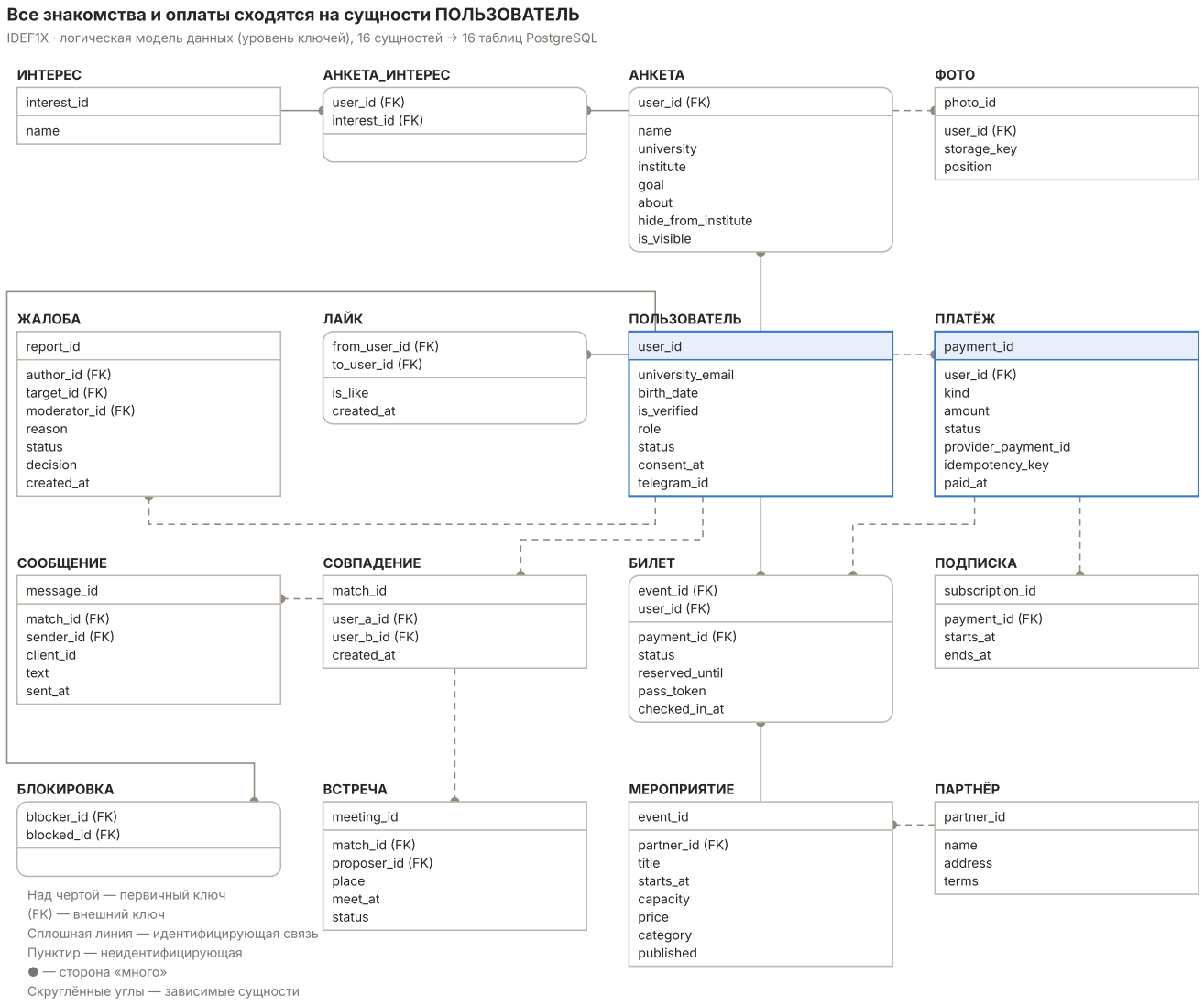
Узел A25 · I1 — предложения партнёров · I2 — заявки студентов · C1 — правила (18+, вместимость, один вход по билету) · C2 — статус оплаты (из A3) · O1 — заказ билета (в A3) · O2 — афиша · O3 — отметки входа и списки участников · M1 — организатор · M2 — сервер CAMPUS

Логическое описание. Организатор (M1) согласует с партнёром дату, вместимость и цену (A251) и публикует мероприятие (A252) — до публикации оно хранится черновиком. A253 принимает заявку студента и проверяет вместимость (C1): если места есть, создаётся бронь на 30 минут, а заказ билета уходит на оплату (O1). Бронь становится билетом с QR-кодом только по статусу оплаты из A3 (C2) — так информация возвращается из направления 3 в направление 2. Неоплаченную бронь студент может отменить сам, а через 30 минут её снимает таймер. A254 отмечает вход: организатор вводит код с QR-билета, и билет засчитывается один раз (O3).

Блок	Вход	Управление	Выход	Механизм
A251 Согласовать с партнёром	I1 Предложения партнёров	C1	Условия площадки	M1
A252 Опубликовать мероприятие	Условия площадки	C1	Мероприятие, O2 Афиша	M1, M2
A253 Записать участников	Мероприятие, I2 Заявки	C1 (вместимость, 18+), C2 Статус оплаты	O1 Заказ билета, билет с QR	M2
A254 Отметить вход по билету	Билет с QR	C1 (один вход)	O3 Отметки входа, списки	M1, M2

8. Модель данных IDEF1X

IDEF1X — расширение IDEF1 для реляционных баз данных: модель показывает сущности, их ключи и связи. Имена атрибутов записаны так, как они называются в таблицах PostgreSQL (схема backend/prisma/schema.prisma).



IDEF1X · модель данных

Логическое описание. В модели 16 сущностей: 11 независимых (прямые углы) и 5 зависимых (скруглённые углы) — АНКЕТА, АНКЕТА_ИНТЕРЕС, ЛАЙК, БИЛЕТ и БЛОКИРОВКА: их первичный ключ состоит из внешних ключей родителей, поэтому без родителей запись существовать не может.

- **ПОЛЬЗОВАТЕЛЬ → АНКЕТА** — связь «один к одному»: ключ анкеты совпадает с ключом пользователя. В пользователе хранятся почта вуза, признак is_verified, статус (активен, скрыт, заблокирован — его меняет модерация) и дата согласия на обработку данных. Признак is_verified лежит именно в пользователе: студент подтверждает почту (A12) до того, как заполнит анкету (A13). Поле telegram_id необязательное — оно понадобится Telegram-боту.

- **АНКЕТА ↔ ИНТЕРЕС** — «многие ко многим», разрешена ассоциативной сущностью АНКЕТА_ИНТЕРЕС. Признак is_visible позволяет студенту временно убрать анкету из ленты.
- **БЛОКИРОВКА** — пара (blocker_id, blocked_id): обе роли ссылаются на ПОЛЬЗОВАТЕЛЯ. Лента, симпатии и чаты исключают заблокированных в обе стороны.
- **ЛАЙК** — зависимая сущность с составным ключом (from_user_id, to_user_id): повторный лайк той же анкеты невозможен на уровне базы. При встречном лайке создаётся СОВПАДЕНИЕ (user_a_id, user_b_id).
- **СОВПАДЕНИЕ → СООБЩЕНИЕ, ВСТРЕЧА** — переписка и встречи существуют только внутри совпадения. client_id сообщения уникален в паре: повторная отправка после обрыва связи не создаёт дубль.
- **ЖАЛОБА** ссылается на пользователя в трёх ролях: автор, нарушитель и модератор.
- **ПАРТНЁР → МЕРОПРИЯТИЕ → БИЛЕТ ← ПОЛЬЗОВАТЕЛЬ** — билет зависит от мероприятия и студента; ссылка на ПЛАТЁЖ появляется после оплаты, pass_token — код для QR, checked_in_at — отметка входа.
- **ПЛАТЁЖ → ПОДПИСКА** — подписка Premium создаётся только из оплаченного платежа; ключ идемпотентности защищает от повторной обработки.

Хранилища DFD (раздел 9) соответствуют таблицам модели: так выполняется правило «БД = хранилища DFD». Номер хранилища начинается с номера направления, которое в него пишет.

Хранилище DFD	Таблицы (сущности IDEF1X)	Задачи дерева, которые пишут
D1.1 Пользователи	ПОЛЬЗОВАТЕЛЬ	11, 12, 143
D1.2 Анкеты	АНКЕТА, АНКЕТА_ИНТЕРЕС, ИНТЕРЕС	13
D1.3 Фото	ФОТО (+ файлы изображений)	13
D1.4 Жалобы / D1.5 Блокировки	ЖАЛОБА / БЛОКИРОВКА	141, 142 / 141
D2.1 Лайки и совпадения	ЛАЙК, СОВПАДЕНИЕ	22
D2.2 Сообщения / D2.3 Встречи	СООБЩЕНИЕ / ВСТРЕЧА	23 / 24
D2.4 Партнёры / D2.5 Мероприятия / D2.6 Билеты	ПАРТНЁР / МЕРОПРИЯТИЕ / БИЛЕТ	251 / 252 / 253, 254, 32
D3.1 Платежи / D3.2 Подписки	ПЛАТЁЖ / ПОДПИСКА	31, 32 / 32

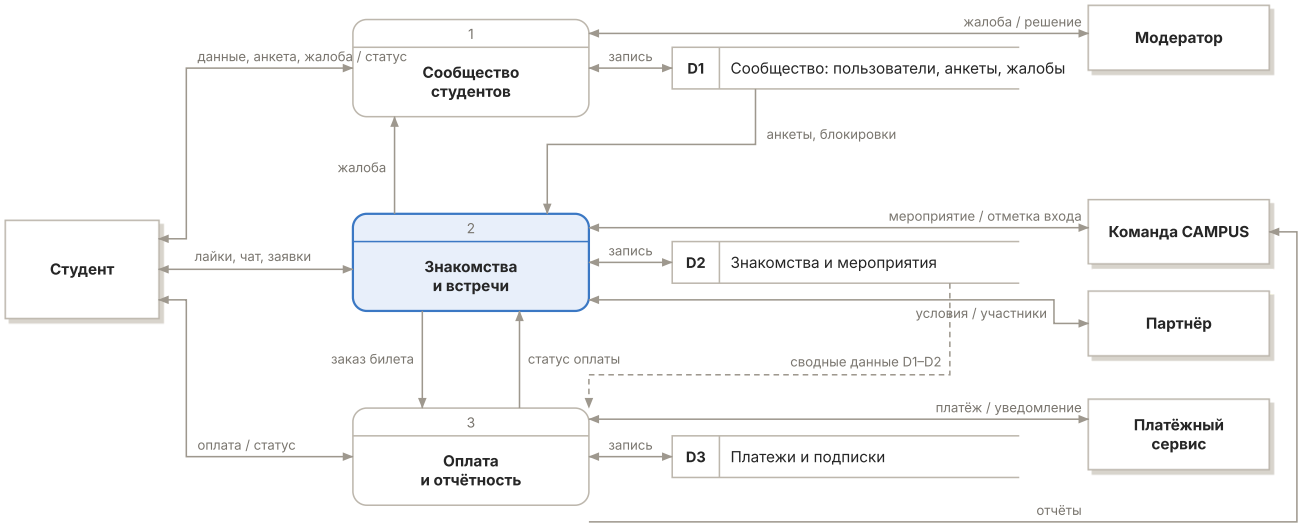
9. Диаграммы потоков данных (DFD)

DFD показывает, какие данные движутся между процессами, внешними сущностями и хранилищами. Процессы 1–3 соответствуют направлениям A1–A3, процессы 11–33 — функциям второго уровня; хранилища — таблицам модели IDEF1X. На первом уровне хранилища показаны укрупнённо (D1–D3), на втором — разделены на таблицы.

9.1. DFD первого уровня

Студент работает со всеми направлениями, а данные между ними передаются через хранилища

DFD · процессы первого уровня (нотация Гейна–Сарсона), хранилища показаны укрупнённо

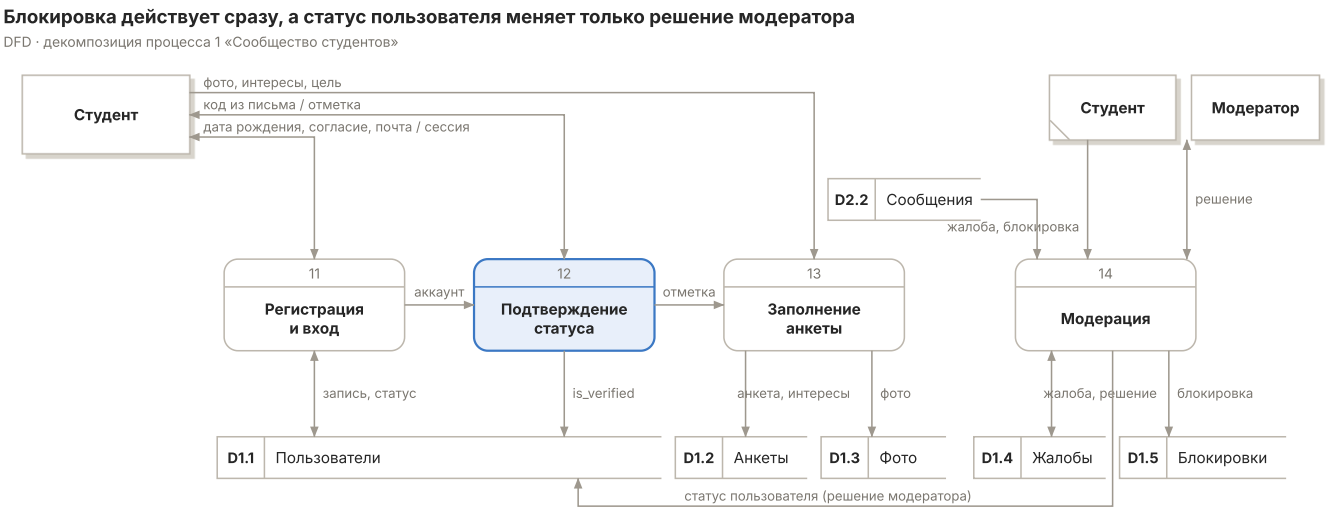


DFD · диаграмма потоков данных первого уровня · скруглённый блок — процесс, прямоугольник с тенью — внешняя сущность, открытый прямоугольник — хранилище

Логическое описание. Внешних сущностей пять: студент, модератор, команда CAMPUS, партнёр и платёжный сервис. Студент работает со всеми тремя процессами, поэтому стоит слева, а остальные сущности — справа, рядом со «своими» процессами. Процесс 1 хранит пользователей, анкеты, жалобы и блокировки в D1. Процесс 2 читает из D1 анкеты и блокировки, пишет лайки, сообщения, встречи и брони в D2 и передаёт жалобы в процесс 1. Процессы 2 и 3 обмениваются заказом билета и статусом оплаты; процесс 3 хранит платежи в D3, получает уведомления платёжного сервиса и отдаёт команде отчёты по сводным данным D1–D2.

Самый нагруженный по данным процесс — «Знакомства и встречи»: он читает анкеты и пишет лайки и сообщения при каждом действии студента. Поэтому таблицы D2 получают индексы по пользователю, а лента отдаётся постранично.

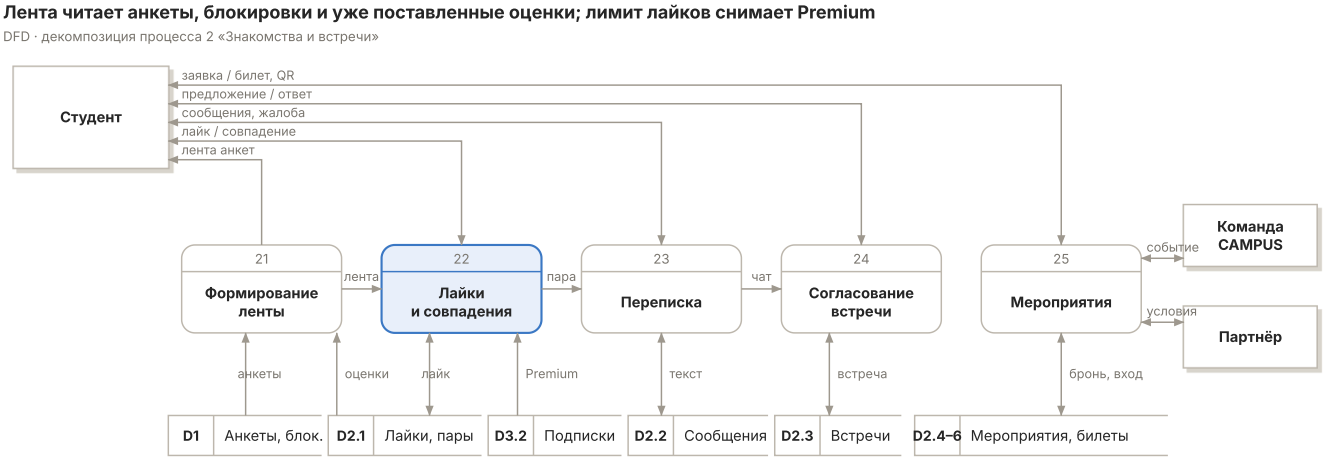
9.2. DFD процесса 1 «Сообщество студентов»



DFD · декомпозиция процесса 1 · срезанный угол — повторное изображение внешней сущности

Логическое описание. Студент передаёт дату рождения, согласие и почту вуза; процесс 11 создаёт или находит запись в D1.1 и проверяет статус (заблокированный не войдёт). Процесс 12 отправляет одноразовый код и после проверки ставит в той же записи отметку is_verified. Процесс 13 сохраняет анкету и интересы в D1.2, фото — в D1.3. Процесс 14 принимает жалобы и блокировки (сущность «Студент» нарисована второй раз, чтобы не тянуть линию через всю диаграмму), читает переписку по жалобе из D2.2, записывает жалобы в D1.4 и блокировки в D1.5. Решение модератора меняет статус пользователя в D1.1 — это единственная запись процесса 14 в чужое хранилище, и именно она реализует петлю обратной связи A14 → A11.

9.3. DFD процесса 2 «Знакомства и встречи»



DFD · декомпозиция процесса 2

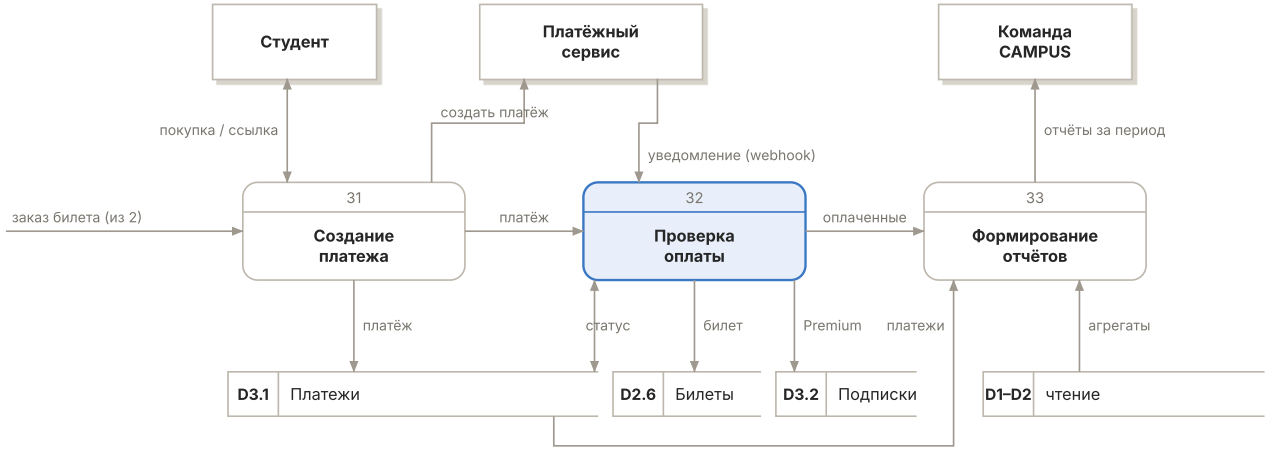
Логическое описание. Процесс 21 собирает ленту: читает анкеты и блокировки из D1 и уже поставленные оценки из D2.1, чтобы не показывать одну анкету дважды. Процесс 22 записывает лайк в D2.1 и читает D3.2, чтобы узнать, действует ли Premium. Процессы 23 и 24 хранят сообщения в D2.2 и встречи в D2.3; из чата студент может пожаловаться. Процесс 25

согласует условия с партнёром, публикует мероприятие вместе с командой, создаёт брони и отмечает вход по QR — всё в хранилищах D2.4–D2.6.

9.4. DFD процесса 3 «Оплата и отчётность»

Платёж создаётся до оплаты, а подтверждается только уведомлением сервиса

DFD · декомпозиция процесса 3 «Оплата и отчётность»



DFD · декомпозиция процесса 3

Логическое описание. Процесс 31 получает заказ билета из процесса 2 (или запрос Premium от студента), создаёт платёж «ожидает» в D3.1 и запрашивает у платёжного сервиса ссылку на оплату. Процесс 32 получает уведомление (webhook), находит платёж в D3.1 и меняет статус; для билета отмечает «оплачен» в D2.6, для Premium создаёт запись в D3.2. Процесс 33 читает платежи и агрегаты из D1–D2 и отдаёт команде отчёт за период.

10. Модели IDEF3

IDEF3 показывает порядок действий внутри сценария: сценарии C1, C2 и C5 из раздела 3 расписаны по функциям второго и третьего уровня (расширенный уровень — не меньше двух диаграмм). Обозначения: прямоугольник — единица работы (UOW) с номером, J — перекрёсток (& — все ветви, X — ровно одна), ELAB — пояснение. Двойная черта слева — сходящийся перекрёсток, справа — расходящийся.

10.1. Сценарий C1 «Регистрация и подтверждение» (A11–A13)



IDEF3 · регистрация и подтверждение статуса студента (A11–A13)

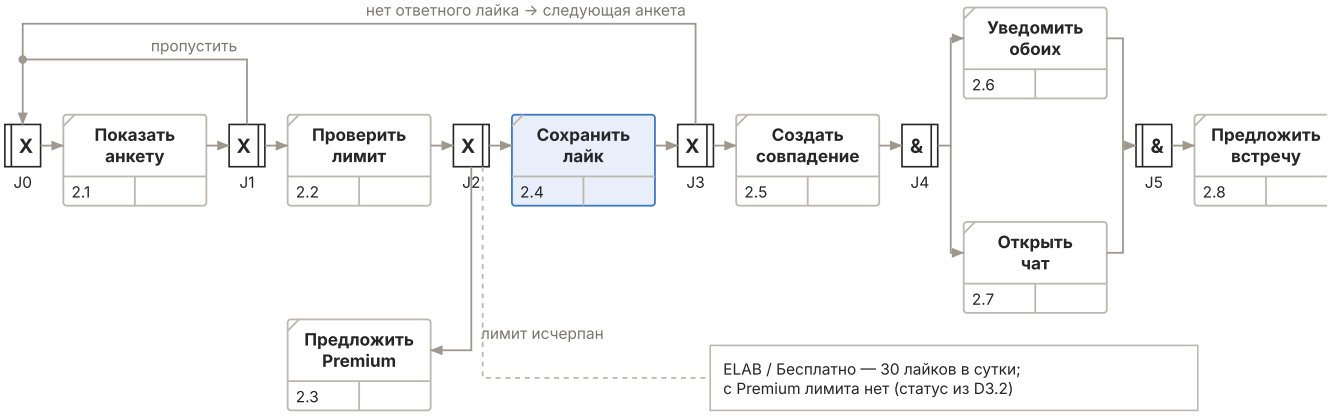
Логическое описание. Студент указывает дату рождения (1.1); перекрёсток J1 проверяет возраст: младше 18 — отказ (1.2). Студент вводит почту вуза и получает код (1.4). Если код неверный, J3 возвращает процесс через J2 к повторной отправке — не больше трёх раз. После верного кода фото и интересы заполняются параллельно (J4 и J5 — &), и анкета публикуется, только когда готовы обе работы.

Перекрёсток	Тип	Смысл
J1	X, расходящийся	18 и старше → ввод почты; младше → отказ (1.2)
J2	X, сходящийся	Отправка кода: первая или повторная после ошибки
J3	X, расходящийся	Код верный → J4; неверный → возврат в J2
J4	&, расходящийся	Фото и интересы заполняются обе
J5	&, сходящийся	Публикация анкеты (1.8) — когда готовы обе работы

10.2. Сценарий C2 «Знакомство» (A21–A24)

Совпадение возникает только при взаимном лайке, иначе студент возвращается к ленте

IDEF3 · сценарий C2 «Знакомство» · блоки A21–A24



IDEF3 · от лайка до предложения встречи (A21–A24)

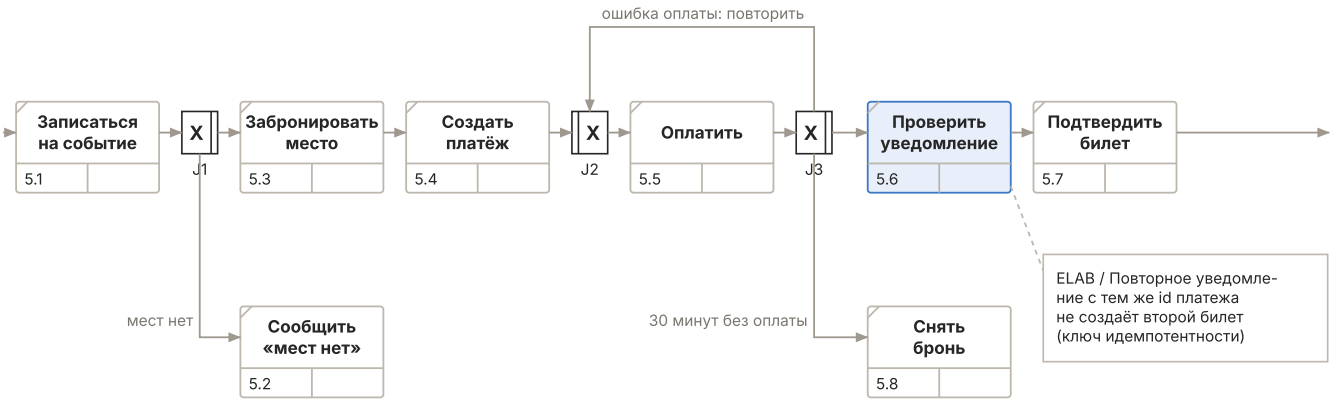
Логическое описание. Студент смотрит анкету (2.1). Перекрёсток J1 разводит действия: «пропустить» возвращает к следующей анкете через J0, «лайк» ведёт к проверке лимита (2.2). Если лимит исчерпан, J2 предлагает Premium (2.3, дальше работает сценарий C6). Лайк сохраняется (2.4), и J3 проверяет встречный лайк: если его нет — следующая анкета, если есть — создаётся совпадение (2.5). Уведомление обоих и открытие чата идут параллельно, после чего можно предложить встречу (2.8).

Перекрёсток	Тип	Смысл
J0	X, сходящийся	Показ анкеты: первый или следующий после пропуска / невзаимного лайка
J1	X, расходящийся	Пропустить → J0; лайк → проверка лимита
J2	X, расходящийся	Лимит есть → сохранить лайк; исчерпан → предложить Premium
J3	X, расходящийся	Встречный лайк есть → совпадение; нет → J0
J4 / J5	&, расходящийся / сходящийся	Уведомление и открытие чата выполняются обе

10.3. Сценарий C5 «Покупка билета» (A253, A31, A32)

Билет подтверждается только после уведомления платёжного сервиса

IDEF3 · сценарий C5 «Покупка билета» · блоки A253, A31, A32



IDEF3 · покупка билета и проверка оплаты (A253, A31, A32)

Логическое описание. Студент записывается на мероприятие (5.1); если мест нет, J1 завершает сценарий сообщением (5.2). Иначе место бронируется на 30 минут (5.3) и создаётся платёж (5.4). После оплаты J3 разводит три исхода: успех ведёт к проверке уведомления (5.6) и подтверждению билета с QR-кодом (5.7), ошибка возвращает к оплате через J2, а если 30 минут прошли без оплаты — бронь снимается (5.8).

Перекрёсток	Тип	Смысл
J1	X, расходящийся	Места есть → бронь; нет → сообщение «мест нет»
J2	X, сходящийся	Оплата: первая попытка или повтор после ошибки
J3	X, расходящийся	Успех → проверка; ошибка → J2; 30 минут без оплаты → снять бронь

11. Требования FURPS+ и сценарии качества

Требования к ИС сгруппированы по модели FURPS+: Functionality, Usability, Reliability, Performance, Supportability и «+» — ограничения. Каждое требование проверяемо и привязано к сценарию (раздел 3) и блоку модели; в последнем столбце — состояние в текущей реализации.

11.1. Требования

ID	Категория	Требование	Сценарий	Блок	Сейчас
F1	Functionality	Вход по одноразовому коду на почту вуза; отказ младше 18; согласие на обработку ПДн	C1	A11	есть
F2	Functionality	Подтверждение статуса студента кодом на почту в домене вуза	C1	A12	есть
F3	Functionality	Анкета: 1–6 фото, 3–10 интересов, цель, «скрыть от своего института», видимость	C1	A13	есть
F4	Functionality	Жалоба и личная блокировка; скрытие при 3 жалобах; решение модератора	C3	A141–A143	есть
F5	Functionality	Лента: только подтверждённые анкеты, без оценённых, скрытых и заблокированных; фильтры возраста, цели, вуза, института	C2	A21	есть
F6	Functionality	Лайк и пропуск; 30 лайков в сутки без Premium; совпадение при взаимном лайке	C2	A22	есть
F7	Functionality	Чат только внутри совпадения; предложение встречи с местом и временем	C2	A23, A24	есть
F8	Functionality	Мероприятие с вместимостью, ценой и черновиком; бронь на 30 минут; QR-билет и однократная отметка входа	C4, C5	A251–A254	есть
F9	Functionality	Оплата билета и Premium в тестовом режиме, подтверждение только по уведомлению	C5, C6	A31, A32	тестовый провайдер
F10	Functionality	Отчёт за период и регистрации по дням; выгрузка CSV	C7	A33	есть
U1	Usability	От первого экрана до первой анкеты в ленте — не больше 7 шагов	C1	A1	9 экранов — сократить
U2	Usability	Интерфейс для телефона (от 360 px): сайт и мобильное приложение; деморежим без сервера	—	A1–A3	есть
U3	Usability	Понятные ошибки: «неверный код, осталось 2 попытки», «лимит лайков исчерпан»	C1, C2	A12, A22	есть
R1	Reliability	Повторное уведомление об оплате не создаёт второй платёж или билет (ключ идемпотентности)	C5	A32	есть
R2	Reliability	Сообщение не теряется при обрыве связи: клиент повторяет отправку с тем же id	C2	A23	есть
R3	Reliability	Резервная копия базы раз в сутки, хранение 7 дней	—	M3	план
P1	Performance	Ответ API ленты ≤ 300 мс для 95% запросов при 60 одновременных пользователях	C2	A21	не замерено
P2	Performance	Новое сообщение доходит до собеседника не дольше чем за 5 с (позже — 2 с через WebSocket)	C2	A23	опрос раз в 5 с

ID	Категория	Требование	Сценарий	Блок	Сейчас
S1	Supportability	Запуск всего сервиса одной командой (Docker Compose)	—	M3	есть
S2	Supportability	Изменения схемы базы только через миграции	—	M3	есть
S3	Supportability	Логи запросов и ошибок; автотесты API для сценариев C1–C7; проверки на каждый pull request	—	A1–A3	есть

«+» — ограничения:

- **Реализация:** backend NestJS (TypeScript) с PostgreSQL (Prisma) и Redis для одноразовых кодов и лимитов; клиенты — сайт (HTML/JS) и мобильное приложение Expo (React Native, TypeScript); Telegram-бот с Mini App — следующий этап. Фото пока хранятся на диске сервера, затем — S3-совместимое хранилище в РФ.
- **Интерфейсы:** REST API в формате JSON, контракт — OpenAPI (backend/openapi.json); API платёжного сервиса в тестовом режиме; SMTP для кодов входа.
- **Физические:** один сервер 2 vCPU / 4 ГБ и хранилище фото от 5 ГБ — из расчёта нагрузки в разделе 3.2.
- **Юридические:** согласие на обработку персональных данных при регистрации (152-ФЗ); хранение данных граждан РФ на серверах в РФ; удаление аккаунта со всеми данными из приложения; только 18+; точная геолокация не собирается; в отчётах — обезличенные данные.

11.2. Сценарии атрибутов качества

Нефункциональные требования расписаны как сценарии из шести частей: источник → стимул → среда → артефакт → отклик → мера отклика.

Сценарий	Источник	Стимул	Среда	Артефакт	Отклик	Мера отклика	Треб.
КС1. Вечерний пик	Студенты	60 человек листают ленту	21:00–23:00	A21, API	Лента отдаётся страницами по 20 анкет по индексу	≤ 300 мс для 95%	P1
КС2. Повторный webhook	Платёжный сервис	То же уведомление пришло дважды	Обычный режим	A32	Второе уведомление распознаётся по id платежа и игнорируется	0 двойных билетов	R1
КС3. Подбор кода	Злоумышленник	10 попыток ввести код подряд	Обычный режим	A12	После 3 ошибок код аннулируется, пауза 15 мин	0 подобранных кодов	F2, U3
КС4. Массовые жалобы	Студенты	3 жалобы на одну анкету за час	Обычный режим	A141	Анкета скрыта из ленты до решения	скрытие ≤ 1 мин, решение ≤ 24 ч	F4
КС5. Новый вуз	Команда	Нужно подключить второй вуз	Разработка	A12, C1	Домен вуза добавляется в настройки, код не меняется	≤ 1 дня работы	S1, S2

12. Модели UML

Диаграммы построены по спецификации UML 2.5.1 консорциума OMG; международный стандарт ISO/IEC 19505 соответствует UML 2.4.1 и с ней совместим. UML дополняет IDEF: IDEF0 и IDEF3 описывают деятельность, а UML — саму ИС: кто ей пользуется, какие данные она хранит и как её части взаимодействуют.

Диаграмма UML	Вид	Что показывает	Связь с другими моделями
Вариантов использования	Поведенческая	Актёров и функции системы с их точки зрения	Актёры — столбцы матрицы ответственности; варианты — листья дерева
Классов	Структурная	Классы предметной области, атрибуты и связи	Совпадает с моделью IDEF1X
Последовательности (три диаграммы)	Поведенческая (взаимодействие)	Обмен сообщениями между частями системы во времени	Сценарии C1, C2, C5; детализация IDEF3

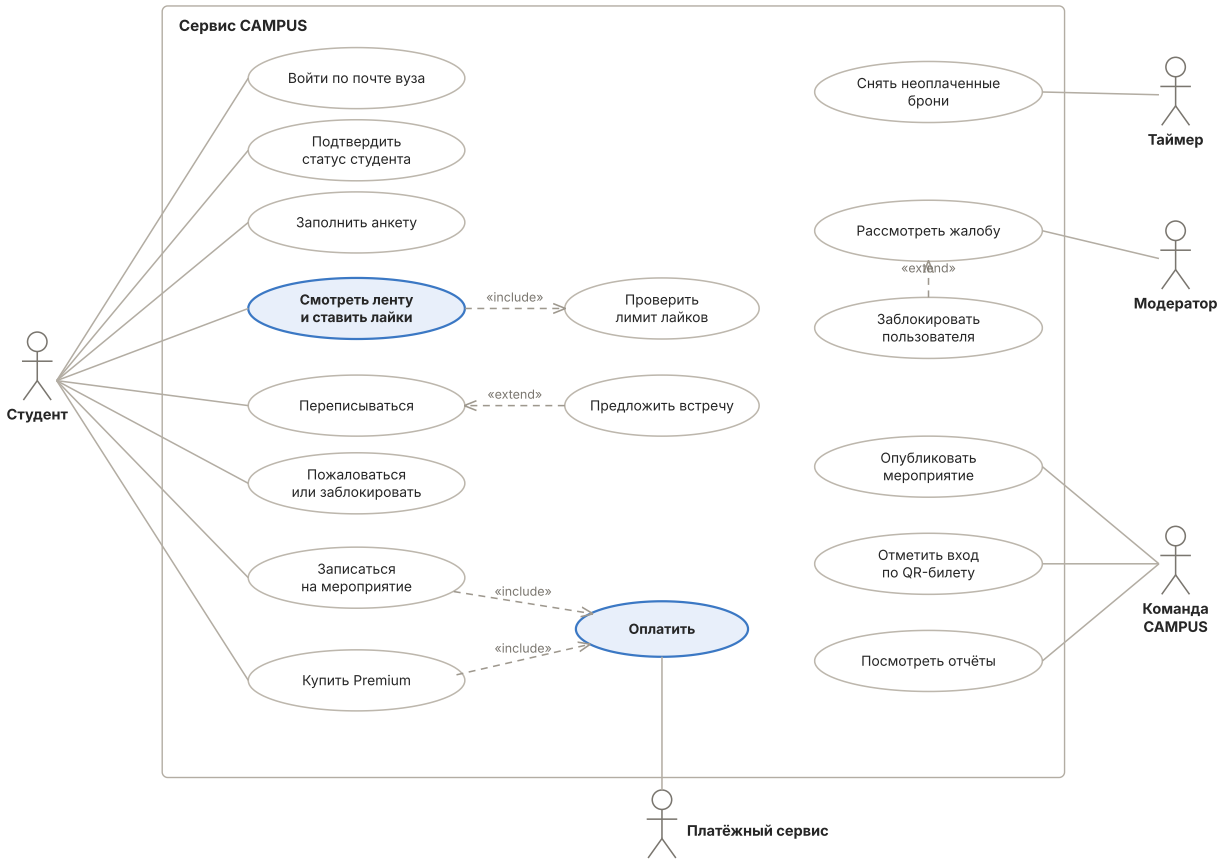
Правила нотации, которые соблюдены:

- Актёры стоят вне границы системы; варианты использования названы глаголом и дополнением.
- «include» — включаемое поведение выполняется всегда, стрелка идёт от базового варианта; «extend» — по условию, стрелка идёт к базовому варианту.
- В классах указаны видимость (+), имя и тип атрибута; у каждой ассоциации — кратность на обоих концах; закрашенный ромб — композиция.
- В диаграммах последовательности синхронный вызов — сплошная линия с закрашенной стрелкой, ответ — пунктир; условия — комбинированный фрагмент alt со сторожевыми условиями в квадратных скобках.

12.1. Диаграмма вариантов использования

Студент — главный актёр; оплата всегда включает проверку платёжным сервисом

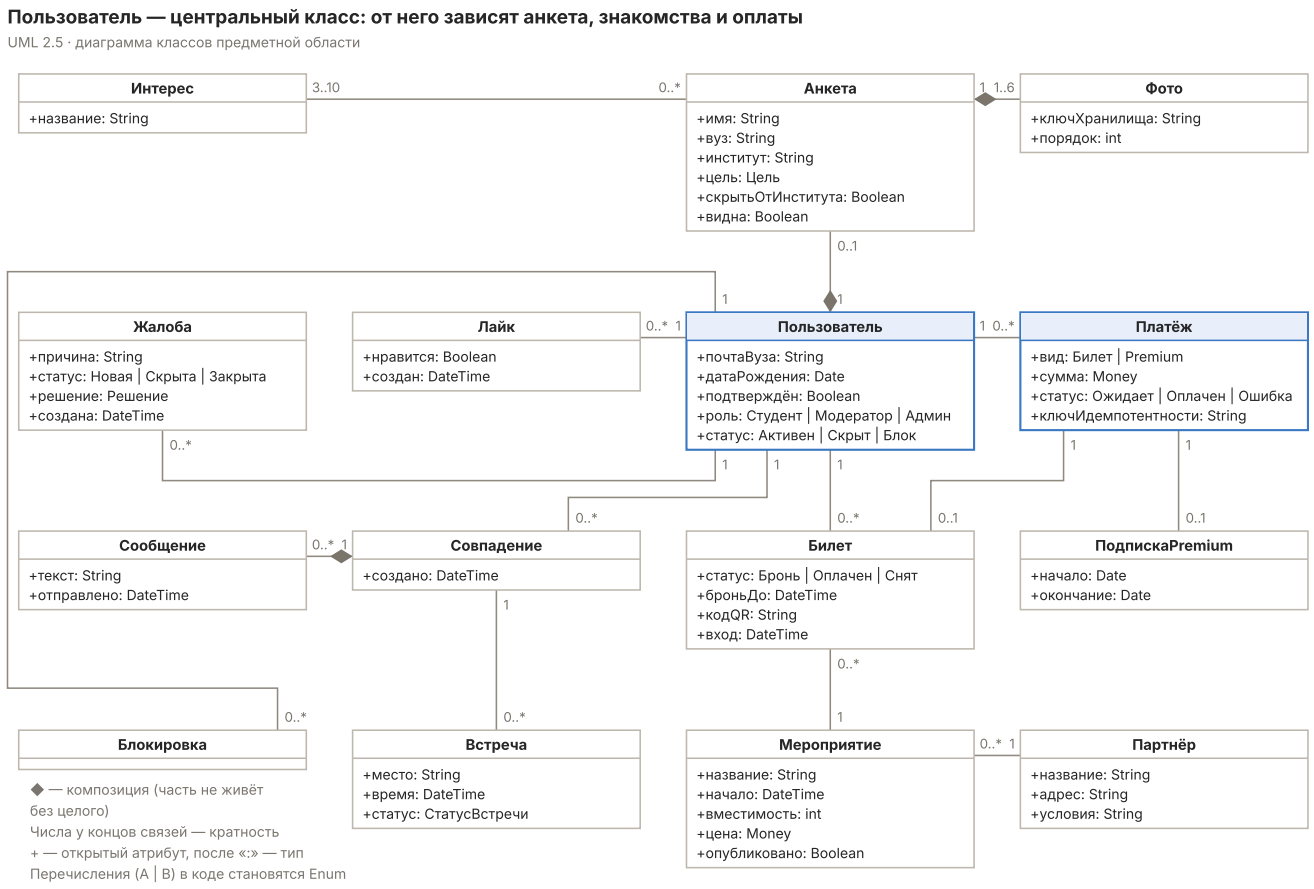
UML 2.5 · диаграмма вариантов использования



UML · диаграмма вариантов использования

Логическое описание. Актёров пять: студент, модератор, команда CAMPUS, платёжный сервис и таймер — системный актёр, который каждые 5 минут снимает неоплаченные брони. Они взяты из столбцов матрицы ответственности, где стоит буква «И». «Смотреть ленту и ставить лайки» всегда включает проверку лимита, а запись на мероприятие и покупка Premium всегда включают оплату («include»). «Предложить встречу» расширяет переписку, а «Заблокировать пользователя» — рассмотрение жалобы: они выполняются только при условии («extend»). Команда CAMPUS публикует мероприятия, отмечает вход по QR-билету и смотрит отчёты. Каждый вариант использования — экран клиента и эндпоинт API (приложение Б).

12.2. Диаграмма классов



UML · диаграмма классов

Логическое описание. Классы соответствуют сущностям модели IDEF1X; ассоциативная сущность АНКЕТА_ИНТЕРЕС на диаграмме классов стала связью «многие ко многим» между Анкетой и Интересом (3..10 интересов у анкеты). Центральный класс — Пользователь: в нём хранятся почта вуза и признак «подтверждён», у него одна Анкета (композиция — анкета удаляется вместе с пользователем), лайки, совпадения, жалобы, блокировки, билеты и платежи. Фото входят в Анкету композицией (1..6), Сообщения — в Совпадение. Перечисления (статусы, роль, вид платежа) задают допустимые значения прямо в модели, а в коде становятся Enum.

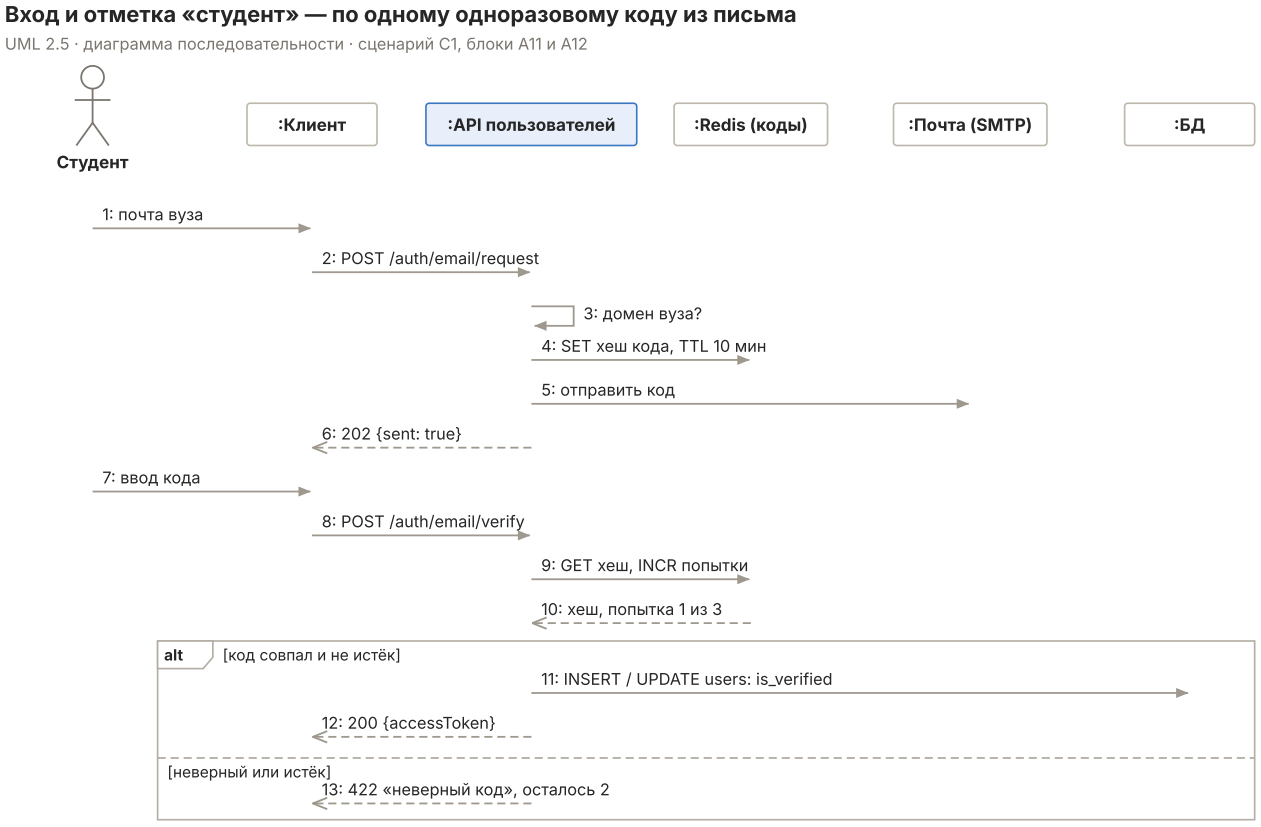
12.3. Диаграмма последовательности: лайк и совпадение (C2)



UML · диаграмма последовательности, сценарий C2

Логическое описание. Студент ставит лайк, клиент (сайт или приложение) вызывает API знакомств (1–2). API проверяет лимит (3–4), записывает лайк и ищет встречный (5–7). Фрагмент alt даёт две ветви: если встречный лайк найден, создаётся совпадение, обоим уходит уведомление и клиент получает match: true (8–10); если нет — match: false (11). Это тот же сценарий, что IDEF3 в разделе 10.2, но на уровне запросов между частями системы.

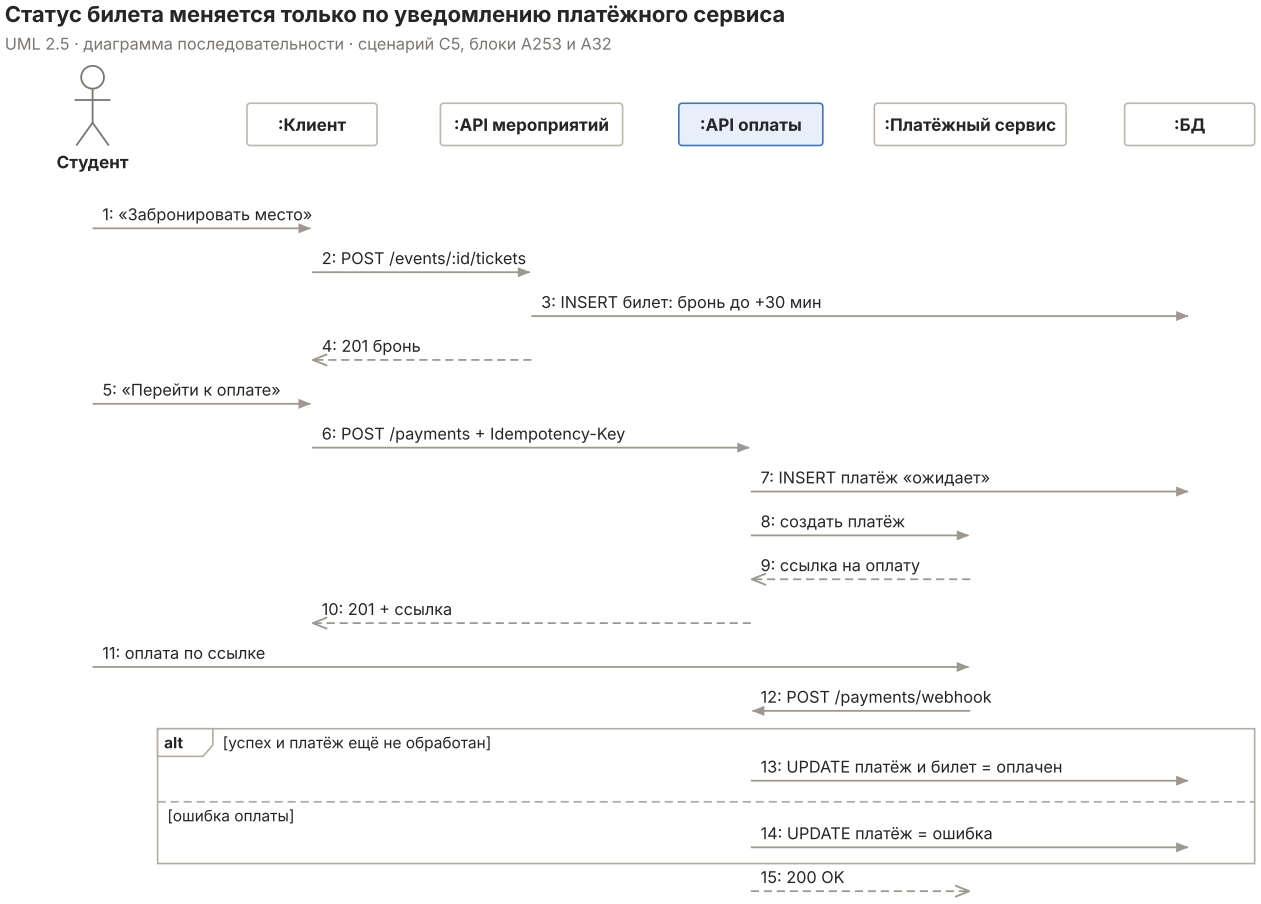
12.4. Диаграмма последовательности: вход и подтверждение статуса (C1)



UML · диаграмма последовательности, сценарий C1

Логическое описание. API проверяет, что домен почты принадлежит вузу (3), сохраняет в Redis только хеш кода со сроком 10 минут (4) и отправляет код письмом (5). При вводе кода API сравнивает хеши и считает попытки (8–10). Если код совпал, запись студента в таблице users (хранилище D1.1) создаётся или обновляется с is_verified = true, и клиент получает токен входа (11–12) — так один код выполняет и A11, и A12. Иначе клиент получает ответ 422 с числом оставшихся попыток (13). Сам код нигде не хранится в открытом виде.

12.5. Диаграмма последовательности: покупка билета (C5)



UML · диаграмма последовательности, сценарий C5

Логическое описание. Студент бронирует место: API мероприятий создаёт бронь на 30 минут (1–4). Затем клиент создаёт платёж с ключом идемпотентности (5–6): платёж записывается со статусом «ожидает», платёжный сервис возвращает ссылку (7–10). Студент платит (11), сервис присылает webhook (12). Фрагмент alt: если оплата успешна и платёж ещё не обработан, статус платежа и билета меняется на «оплачен» (13); при ошибке платёж помечается ошибкой (14). В любом случае сервис получает 200 OK (15), чтобы не присылать уведомление снова.

13. Словарь стрелок и выводы

13.1. Словарь стрелок (диаграмма A0)

Стрелка	Тип	Откуда → куда	Содержание
I1 Данные студента	I	граница → A1	Дата рождения, согласие, почта вуза, код, фото, интересы
I2 Действия студентов	I	граница → A2, A3	Лайки, сообщения, жалобы, заявки, покупка Premium
I3 Предложения партнёров	I	граница → A2	Площадка, даты, вместимость, условия
I4 Уведомления платёжного сервиса	I	граница → A3	Результат оплаты (webhook)
C1 Правила сервиса и модерации	C	граница → A1, A2	18+, лимит лайков, правила общения и мероприятий
C2 152-ФЗ «О персональных данных»	C	граница → A1, A3	Согласие, хранение в РФ, обезличенные отчёты
C3 Тарифы и условия оплаты	C	граница → A3	Premium 349 ₽ на 30 дней, бронь 30 минут
Анкеты и статусы	O→I	A1 → A2	Подтверждённые анкеты и статусы пользователей с учётом модерации
Жалобы и блокировки	O→I (обратная связь)	A2 → A1	Жалоба: автор, нарушитель, причина; личная блокировка
Заказ билета	O→I	A2 → A3	Бронь: мероприятие, студент, сумма
Статус оплаты	O→C (обратная связь)	A3 → A2	Оплачен / ошибка; Premium до даты
O1 Совпадения, переписка, встречи	O	A2 → граница	Результат знакомства для студента
O2 Мероприятия и билеты	O	A2 → граница	Афиша, QR-билеты, отметки входа
O3 Уведомления	O	A1, A3 → граница	О решении модератора и об оплате
(Сводные данные D1–D2)	I, туннельная	хранилища D1–D2 → A3 (A33)	Регистрации, совпадения, встречи, билеты — для отчёта
O4 Отчёты	O	A3 → граница	Регистрации, совпадения, встречи, выручка за период
M1 Модератор	M	граница → A1	Рассмотрение жалоб (A142)
M2 Организатор мероприятий	M	граница → A2	Мероприятия и отметка входа (A25)
M3 Сервер CAMPUS и клиенты	M	граница → A1–A3	NestJS, PostgreSQL, Redis; сайт и мобильное приложение

13.2. Выводы

1. Деятельность сервиса раскрыта сверху вниз: цель → три направления (A1–A3) → 11 функций → 7 задач третьего уровня у модерации и мероприятий. Каждое разбиение содержит 3–5 узлов, как требует IDEF0, а дерево растёт от одного ствола, а не «кустом» от корня.
2. Направления совпадают с модулями программы: сообщество — users и moderation, знакомства — matching и events, оплата — payments; 17 листьев дают список эндпоинтов API (приложение Б).
3. Две петли обратной связи — жалобы и решения модерации (A2 → A1 → A2) и статус оплаты (A3 → A2) — повторяются в IDEF0, DFD, IDEF3 и диаграммах последовательности.
4. Каждый лист дерева пишет в своё хранилище; хранилища DFD совпадают с таблицами IDEF1X и классами UML (16 таблиц, из них 5 зависимые).
5. Матрица ответственности показала, что студент исполняет 9 задач из 17, а задачи 21, 32 и 33 выполняются автоматически, — отсюда актёры диаграммы вариантов использования.
6. Нагрузка пилота небольшая (≈ 60 пользователей онлайн в пик), всё помещается на один сервер; главный риск — не производительность, а плотность аудитории и безопасность пользователей.

Приложение А. Техническое задание и состояние реализации

ТЗ сформировано по модели: модули — направления и функции, эндпоинты — листья дерева, таблицы — сущности IDEF1X, требования — FURPS+. В столбце «Состояние» отмечено, что уже сделано в репозитории проекта.

А.1. Назначение и пользователи

Сервис знакомств студентов одного вуза (пилот — УрФУ) с чатом, предложениями встреч, мероприятиями и тестовой оплатой. Один API, три клиента: сайт, мобильное приложение и Telegram-бот с Mini App. Роли и права — по матрице ответственности (раздел 1.3): студент, модератор, администратор (он же организатор).

А.2. Модули, эндпоинты и таблицы

Направление · модуль	Эндпоинты REST API	Таблицы	Задачи	Состояние
A1 · users	POST /auth/email/request · POST /auth/email/verify · GET, PUT, DELETE /profile · POST /profile/photos/upload · PATCH /profile/photos/order	users, profiles, photos, interests, profile_interests	11–13	сделано
A1 · moderation	POST /reports · POST, DELETE /users/:id/block · GET /admin/reports · POST /admin/reports/:id/decision	reports, user_blocks, users.status	141–143	сделано
A2 · matching	GET /feed · POST /likes · GET /matches · GET, POST /matches/:id/messages · POST /matches/:id/meetings · PATCH /meetings/:id	likes, matches, messages, meetings	21–24	сделано
A2 · events	GET /events · POST, PATCH /admin/events · POST /admin/partners · POST, DELETE /events/:id/tickets · GET /tickets/:id/pass · POST /admin/tickets/check-in	partners, events, tickets	251–254	сделано
A3 · payments	POST /payments · GET /payments/:id · POST /payments/webhook · GET /subscription · GET /admin/stats · GET /admin/stats/daily	payments, subscriptions	31–33	сделано, провайдер тестовый

А.3. Требования и ограничения

- Функциональные, качественные и юридические требования — таблица FURPS+ (раздел 11); каждое требование закрывается автотестом или ручной проверкой по сценарию.
- Безопасность: секреты — только в переменных окружения. Доступ проверяется на сервере, а не скрытием в интерфейсе: анкету видит тот, кому она разрешена в ленте (владелец подтверждён, не скрыт, не заблокирован, нет личной блокировки, зритель не из его института при включённом скрытии); переписку и встречи видят только участники совпадения (и модератор — только переписку по жалобе).
- Фоновая задача (таймер): каждые 5 минут снимать брони старше 30 минут. Premium считается активным, пока не прошла дата окончания.

А.4. Этапы работ

Этап	Результат	Задачи дерева	Состояние
1	Каркас API, схема БД и миграции, вход по почте вуза, анкета и фото	11–13	сделано
2	Лента, лайки, совпадения, чат, встречи, жалобы и блокировки	21–24, 141–143	сделано
3	Мероприятия, QR-билеты, тестовая оплата, отчёты	251–254, 31–33	сделано
4	Сайт и мобильное приложение (APK собирается автоматически)	все	сделано
5	Telegram-бот с Mini App, push-уведомления, реальная оплата, сервер в интернете	11, 32, М3	план

А.5. Критерии приёмки

- Два тестовых студента проходят сценарии С1 и С2 до предложения встречи без вмешательства разработчиков.
- Жалоба от трёх пользователей скрывает анкету; решение модератора меняет статус пользователя (С3).
- Повторное уведомление об оплате не создаёт второй билет; неоплаченная бронь снимается через 30 минут (С5).
- Выполнены требования Р1 и R1–R3; сервис поднимается одной командой.

Приложение Б. От модели к коду

Структура репозитория повторяет модель: направления дерева — модули сервера, листья — эндпоинты, хранилища — таблицы базы. Клиенты не содержат бизнес-правил: всё, что описано в блоках IDEF0, проверяет сервер, поэтому сайт, приложение и будущий бот ведут себя одинаково.

Уровень модели	Где в репозитории	Что там
Направления A1–A3, функции A11–A33	backend/src/users, moderation, matching, events, payments	Модули NestJS: контроллер (эндпоинты), сервис (правила), DTO (проверка данных)
Хранилища D1–D3 (IDEF1X)	backend/prisma/schema.prisma, migrations/	Таблицы PostgreSQL и миграции
Механизм M3	backend/Dockerfile, docker-compose.yml	Сервер, база, Redis и том для фото — одной командой
Контракт между сервером и клиентами	backend/openapi.json, docs/API.md	Описание всех запросов и ответов
Клиент: сайт	frontend/web	Экраны по 7 листам макетов, деморежим
Клиент: мобильное приложение	frontend/mobile	Expo / React Native, APK в разделе Releases
Клиент: Telegram	frontend/telegram	План: бот и Mini App, вход по Telegram
Проверка сценариев C1–C7	backend/test, .github/workflows	Сквозные тесты на настоящей базе на каждый pull request